Technical Documentation

SEO Fixer · 30 October 2026 · Sohan Reddy

Setup and running

SEO Fixer is a Django app that runs on your own machine. It takes 7 steps to set up, all from the project folder.

1. Create and turn on a virtual environment: `python3 -m venv .venv` then `source .venv/bin/activate`
2. Install packages: `pip install -r requirements.txt`
3. Install the browser: `python -m playwright install chromium`
4. Copy `.env.example` to `.env` and set `DJANGO_SECRET_KEY` to any random string. This is the only required key.
5. Create the session database: `python manage.py migrate`
6. Start the app: `python manage.py runserver`, then open `http://localhost:8000`
7. Click **Build / refresh SEO knowledge base** once. This downloads 12 Google guide pages into the local search index (no API key needed).

Add any other keys to `.env`, then restart the server for them to take effect.

Configuration

Only `DJANGO_SECRET_KEY` is required. Each other key turns on one feature.

`.env` key	Needed?	What it turns on
`DJANGO_SECRET_KEY`	Yes	Signs the session cookie and CSRF tokens
`ANTHROPIC_API_KEY`	No	The 4 AI agents and their suggested fixes
`GITHUB_TOKEN`	No	Creating draft pull requests
`PAGESPEED_API_KEY`	No	Google's SEO, accessibility, performance and best-practices scores, page weight and load time
`LLM_MODEL`	No	Which Claude model the agents use (default `claude-haiku-5-5`)

These fixed values live in `seofixer/settings.py`:

Setting	Value	Meaning
`AGENT_MAX_TURNS`	3	Most AI rounds per agent
`LLM_MAX_TOKENS`	4000	Longest AI reply
`MAX_FIXES`	10	Most fixes shown after merging
`MAX_BODY_CHARS`	5000	Page text sent to the AI
`SEARCH_RESULTS`	3	Guide passages returned per search
`SESSION_COOKIE_AGE`	86400	An audit is kept for 1 day
`SEO_SOURCES`	12 URLs	Google guide pages in the search index
`WEB_SEARCH_DOMAINS`	5 sites	Sites the agents may web-search
`CHROMA_DIR`, `SCREENSHOT_DIR`	folders	Where the index and screenshots are saved (both gitignored)

Files

All app code is in `seofixer/`, about 1,800 lines in total. Start reading at `views.py`.

File	Lines	What it does
`views.py`	101	The one page: reads the button pressed and calls the right module
`audit.py`	178	Checks the URL is public, opens the page in a browser, runs the 11 rule checks
`measurements.py`	177	Google scores, image checks, before/after table, screenshot difference
`ai.py`	235	Builds and searches the guide index; runs the 4 agents in parallel and merges results

`prompts.py`	105	The agents' roles, rules and tool definitions (change AI wording here)
`github.py`	213	Applies chosen fixes, shrinks images, opens one draft PR
`templates/seofixer/index.html`	209	The page layout and its 4 buttons
`settings.py`	80	All settings and keys
`tests.py`	407	Automated tests
`urls.py`, `asgi.py`, `wsgi.py`, `manage.py`	small	Standard Django wiring

How an audit runs

Each of the 4 buttons sends one `action` to `views.home`. The audit itself goes through these steps:

1. The URL is checked: only public `http`/`https` addresses on port 80 or 443.
2. Chromium opens the page at 1280×800, scrolls to the bottom, measures up to 60 images, and takes a screenshot. It then checks image widths again at 1920×1080. Google PageSpeed runs at the same time.
3. Beautiful Soup reads the title, meta tags, headings, links and alt text, and the 11 rule checks run.
4. Four agents run in parallel. Each searches the guide index first (up to 3 searches), may web-search, then submits up to 4 fixes:
   - Titles and snippets agent
   - Content agent
   - Images and accessibility agent
   - Links and technical agent
5. Each fix is checked: its source must come from the agent's searches, and its text must come from the page.
6. Results are merged, most severe first, with duplicates removed (10 at most), and saved in the session.
7. When you open a PR, each fix is applied only if its exact text appears once in the repo. Everything goes into one commit on a new `seo-fixes-…` branch as a draft PR. The PR stays a draft for review.
8. **Measure again** re-reads the page and shows a before/after table and a screenshot with changed pixels in red.

Tests

Run all 31 tests with `python manage.py test seofixer`. No API keys are needed: Claude, GitHub and PageSpeed are simulated, while HTML parsing, Chroma search, image processing and page requests are real.

Test group	Tests	What it covers
`ParsingTests`	7	Reading pages, rule checks, URL safety, repo names
`AgentTests`	8	Agent loop, citation checks, web search, parallel runs
`ImageAndEditTests`	7	Image resizing, exact-once edits, screenshot difference
`RetrievalTests`	1	Real Chroma search and removal of old chunks
`GitHubTests`	3	One commit, draft PR, repo listing checks
`PageFlowTests`	5	CSRF, keyless audits, form safety, re-measuring

Operating notes

- The app is designed to run locally on each developer's machine.
- AI fixes use `ANTHROPIC_API_KEY` and the guide index. Rule checks and measurements always run.
- PRs use `GITHUB_TOKEN` and the repo entered with the audit. Each audit opens one draft PR.
- Supports repos with up to 150 editable files of up to 200 KB each.
- Image attributes are edited in `.html` and `.htm` files.
- An audit is kept for 1 day, or until a new audit starts.
- Full details of every operation are logged in the server terminal.